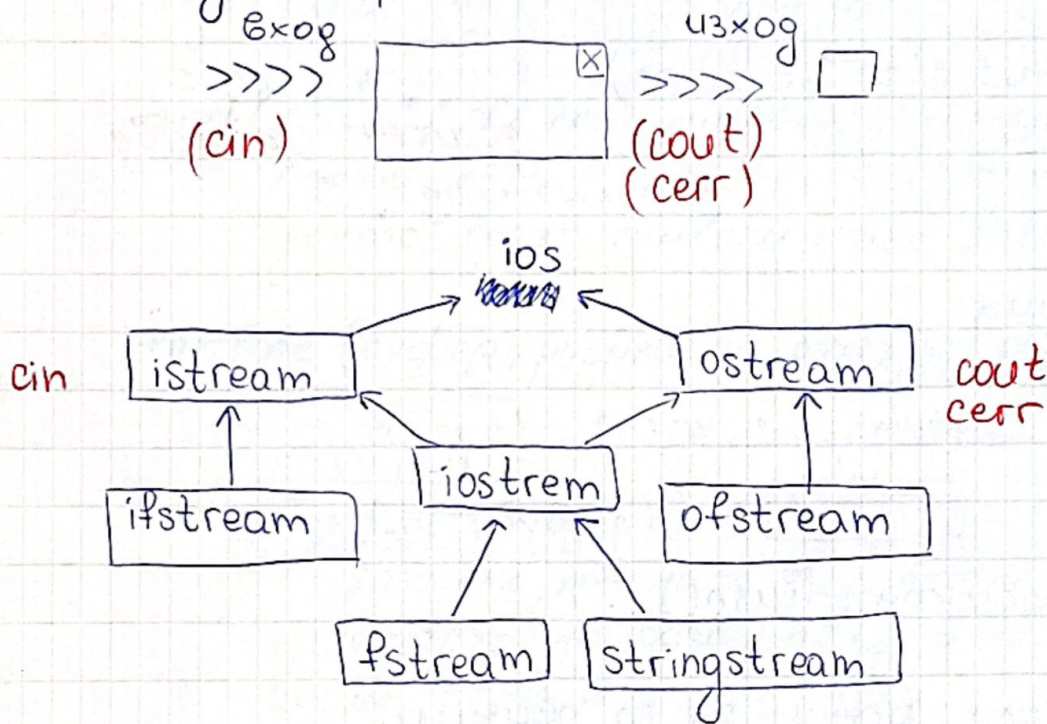


05.03.2024г.

Потоци и файлове

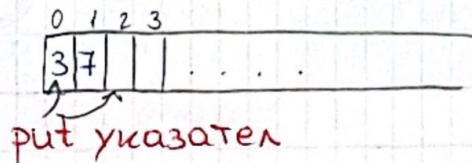


`ifstream ifs("test.txt")` $\longrightarrow$ `.open(...)` // публична функция

`}` унищожава се `ifs` $\longrightarrow$ `.close(...)` // публична функция. Хубаво е да я викаме, но и да забравим не е грешка

Интерфейс:

Потоци за изход:
// имамо масив без край



$\rightarrow$ Форматиран изход
`ostream << обект`

символ $\leftarrow$ то символ
"поставя обекта"

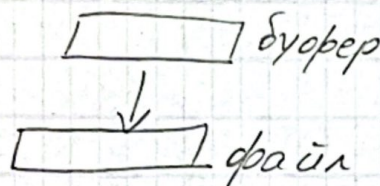
Неформатиран

- `put(char ch)` // поставя един символ и мести `put` указателя с 1
- `write(const char *str, size)` // използва се с `потегатва string` **двоични файлове**
// нямаме `"\0"`

Синхронизация:

Имаме буфер

- `flush()` // функция, която



`close()` извиква `flush()`

`Put` указателя можем да го двитим.

Позициониране:

- `tellr()` - на кой индекс е `put`
// ако всичко е както трябва, ще е 0
- `seekr(...)` - преместване на указателя
приема

Потоци за вход:

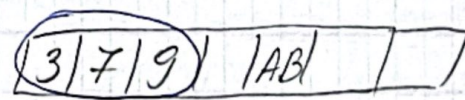
форматиран вход

`ifs >> <обект>`

// има краен масив

`(int)n;`

`ifs >> n`



get указател

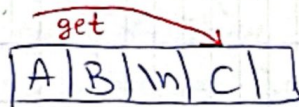
// ако дадем `int`, а искаме друг тип, потока се сун

неформатиран

- get() // връща 1 символ // връща int
→ get(char *buff, брой символи) // void function
разделител по default е „\n“

→ get(char *buff, брой символи, ^{char}разделител)
// четем рокато срещнем разделител или
докато броят символи стане равен

→ getline (буфер, размер на буфера, разделител = '\n')



getline (буфер) → 

// get указателя прескача разделителя

// на мястото на разделителя в новия масив
имаме терминираща нула

Позициониране

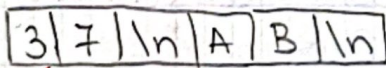
→ tellr() - индекс на get

→ seekg(...) - къде да преместим указателя
приема параметри

→ ignore(брой символи // по default е 1, разделител //
по default е eof)

// мести get указателя





int n; } // не прескачаме новия ред

cin >> n; } cin.ignore()

char buff[1024];

cin.getline(buff, 1024)

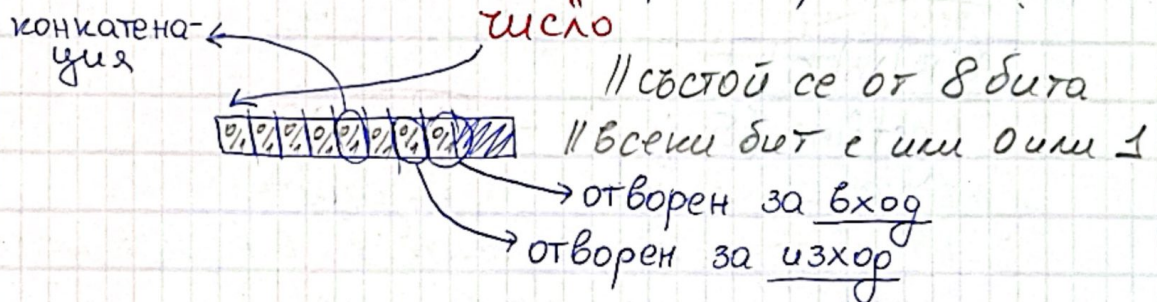
// в буфера се записва AB \0

peek() - връща текущия символ и не мести
get указателя

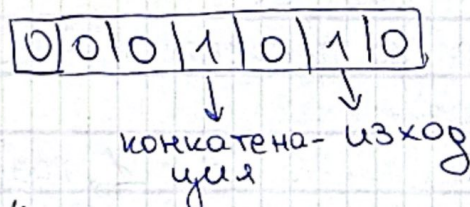
fstream / stringstream - потоци
• съдържат всички функции

Решим на работа

ofstream ofs (име, решим на работа)



Пример ofstream ofs ("text.txt", 10);



// слепваме, не маскираме информацията
във файла, а слепваме старото с новото

constant:

ios::in - поток отворен за вход 1
ios::out - отворен за изход 2
ios::ate - отваря потока и поставя
указателя накрая 4

// позволява позициониране

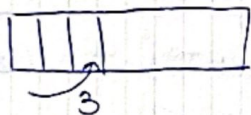
можем да се върнем назад и да заучаваме

`ios::app` - отваря потока за изход и 8
поставя `put` указателя накрая
// не позволява да се връщаме назад
// не позволява позициониране

`ios::trunc` - Ако файлът съществува, то той се 16
изтрива // има смисъл при
поток за изход

`ios::binary` - двоичен режим 32
`ofstream ofs ("A.txt", ios::out | ios::app);`
↑ побитово "или"
10
`fstream`

→ Позициониране
`tellp()` / `tellg()`



`seekg(3, index)` // за `seekp` е същото
`seekg(offset, pos)`

`pos : ios::beg`

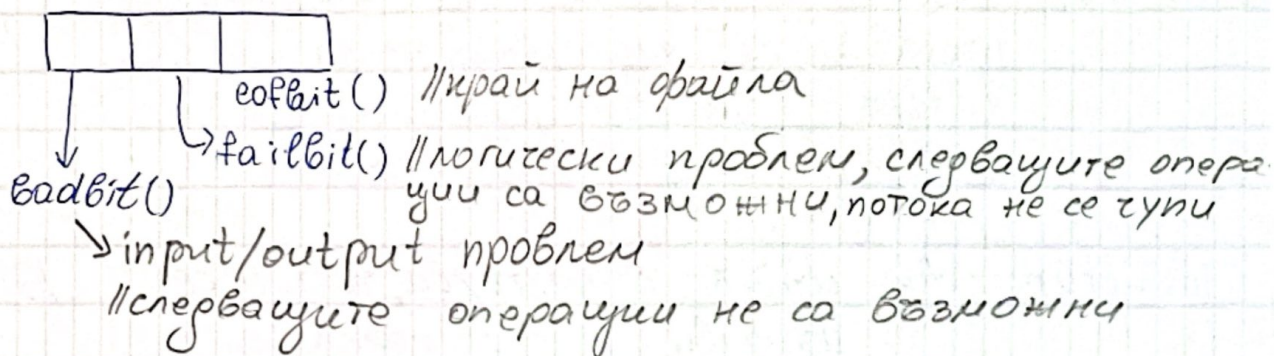
`ios::end`

`ios::cur`

// праща те след последния символ

Забелка: Функция, която приема име на файл и връща големината му в байтове.

Състояние на поток



Имаме функции:

fail() → ако badbit или failbit е вригнат

bad() → badbit е вригнат

eof() → eofbit

good() → когато и трите бита са 0

clear() → изчиства състоянието на потока

Ако искаме да разберем failbit правим отрицанието на badbit.

stringstream

```
stringstream str("37AB");
```

```
int x;
```

```
str >> x;
```

```
str << 79;
```

```
(str.str()) // връща const char *
```

името на string

CSV parser

→ comma separ values

Текстов файл, който е таблица

A	B
x	y

→ .txt

A, B — имена на колоните

x, y — ред 1

Първо : зареждане в паметта
поле - стринг
ребр - масив от ~~ребра~~ полета
таблица - масив от реброве

Двоични файлове



"def": Готов за ~~редактиране~~
зареждане в паметта "

число → стринг
стринг → число

Текстов файл [712]

712 → двоичен файл [AF|17|00|00]

в паметта

[AF|17|00|00] // 16-та бройна система = 712

Текстов файл - лесен за нас, но е
труден за програмата;
Двоичен файл - лесен за програмата, но
труден за нас // за четене

• write (масив от байтове, колко елемента да запише)

int x = ...;

ofs - поток за изход

ofs.write ((const char*) &x, sizeof(int)) // може и sizeof(x)

const char*

int*

4

! Защо са избрани const char*, а не int*
char ≈ байт // за теория

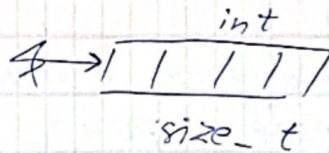
Четене:

```
ifs (... , ios::binary)
```

```
int x;
```

```
ifs.read((char*) &x, sizeof(x))
```

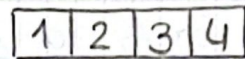
къде да постави данните, колко данни да прочете



```
cout << x; // числото, което сме запазили във файла
```

Масив от числа

```
int arr[] = {1, 2, 3, 4};
```



```
ofs.write((const char*) arr, sizeof(arr))
```

големината на масива * sizeof(int)
// в нашия случай = 16

// с ерин write записваме цял масив,
защото имаме последователност от полета
// arr, не се поравя с &, защото вече
имаме &arr в самото име arr

Прочитане

```
int arr[4];
```

```
ifs.read((char*) arr, 4 * 4)
```

големината на масива
sizeof(int)
// ще прочетем ~~ще бъде~~ данните от файла

```
struct X
```

```
{ bool b;
```

```
int n;
```

```
}
```

```
X obj;
```

```
write((const char*) &obj, sizeof(X))
```

8